

Preventia360 External API (v1)

Integrator Guide — External Data API

Audience: engineers building third-party integrations (bots, CRMs, ops tools) against the Preventia360 External API. You will receive an API key from a Preventia360 administrator — store it in your own secrets vault; it cannot be shown again.

Overview

Read-only HTTP API for third-party applications (bots, CRMs, ops tools) that need to render Preventia's gig booking forms and pricing outside our own apps — for example, a WhatsApp bot that walks a customer through booking a sahayak visit.

It serves exactly what our own provider app renders: the same roles, service categories, dynamic PRE/DURING/POST forms, and pricing. There is no write access — bookings themselves still go through the regular app/backoffice flow.

The API is authenticated with an API key issued to your integration by a Preventia360 administrator. Your integration only ever sends the raw key — there is nothing else to configure on your side.

Base URL & response envelope

```
https://<your-backend-host>/api/v1
```

(Backend context-path is `/api/v1`; every path below is relative to that.)

Every successful call — on both APIs — returns the same envelope:

```
{
  "success": true,
  "data": { "...": "..." },
  "timestamp": "2026-10-01T12:34:56.789Z"
}
```

Every handled error returns:

```
{
  "success": false,
  "errorCode": "ROLE_OUT_OF_SCOPE",
  "message": "This API key is not scoped for role 'DOCTOR'.",
  "messageKey": "errors.ROLE_OUT_OF_SCOPE",
  "timestamp": "2026-10-01T12:34:56.789Z"
}
```

Exception — authentication failures. A missing, malformed or revoked bearer key does **not** go through that JSON envelope — it's rejected at the security-filter layer before any controller runs, as a bare `403 Forbidden` with an empty body. Treat any `403` with no JSON body as "the credential wasn't accepted"; a `403` with a JSON body (`errorCode` present) is a business-rule rejection like `ROLE_OUT_OF_SCOPE` instead.

API reference

Base path: `/external/v1`. Every request needs the API key issued to your integration:

```
X-API-Key: pv_live_8f2a91c4k3nQ7z1xR5pL9mW2vY8bT4sU6hC0eF3gD1aN7jK5i
```

Authorization: `Bearer <key>` works identically if that header is more convenient for your HTTP client. Don't send both an admin JWT and an API key on the same request — this is a separate credential type.

Status	Meaning
403 (empty body)	Missing, malformed, or revoked key
403 <code>ROLE_OUT_OF_SCOPE</code> (JSON body)	Key isn't scoped for the <code>role</code> you asked about
400 <code>UNKNOWN_ROLE</code>	<code>role</code> isn't an active provider role
400 <code>ROLE_CATEGORY_MISMATCH</code>	<code>serviceCategoryId</code> doesn't belong to the given <code>role</code>
404	<code>serviceCategoryId</code> doesn't exist

2.1 List roles

```
GET /external/v1/roles
X-API-Key: pv_live_...
```

Every active **provider** role this key may query (already filtered to the key's scope — an unscoped key sees all of them).

Response

```
{
  "success": true,
  "data": [
    { "code": "SAHAYAK", "label": "Sahayak", "spawnStrategy": "GIG" },
    { "code": "DOCTOR", "label": "Doctor", "spawnStrategy": "CONSULTATION" }
  ],
  "timestamp": "2026-10-01T12:10:00Z"
}
```

`spawnStrategy` tells you the shape of the booking lifecycle: `GIG` roles (Sahayak, future field-service roles) have PRE/DURING/POST stage forms as documented in §2.4; `CONSULTATION` and `ORDER` roles don't use the gig-forms endpoint.

2.2 List service categories (+ pricing)

```
GET /external/v1/service-categories?role=SAHAYAK
X-API-Key: pv_live_...
```

Every active service category for that role, each with its base pricing embedded — this is usually your starting call after `/roles`.

Response

```
{
  "success": true,
```

```

"data": [
  {
    "id": "a2ee489a-c7aa-3424-e65a-8fec37beee6c",
    "code": "VITALS",
    "providerType": "SAHAYAK",
    "name": "Vitals",
    "description": null,
    "pricing": {
      "configured": true,
      "pricingType": "DYNAMIC",
      "currency": "INR",
      "baseFare": 200.00,
      "hourlyRate": 100.00,
      "perDayRate": null,
      "perMonthRate": null,
      "perKmRate": null,
      "overageRate": null,
      "submissionFee": 0.00,
      "surchargeMultiplier": 1.00,
      "freeTier": 60,
      "billingCap": 150,
      "workingHoursPerDay": 9,
      "locationZone": null
    }
  },
  {
    "id": "177e5dad-0fc0-a6f2-5e93-cd214d9ae9e0",
    "code": "OUT_PATIENT_VISIT",
    "providerType": "SAHAYAK",
    "name": "Out Patient Visits",
    "description": null,
    "pricing": { "configured": false }
  }
],
"timestamp": "2026-10-01T12:11:00Z"
}

```

Keep each `id` — it's `serviceCategoryId` in §2.3 and §2.4. `pricing.configured: false` means no pricing rule has been set up yet for that category (every other pricing field is omitted, as in the second row above).

2.3 Pricing (standalone refresh)

```

GET /external/v1/pricing?role=SAHAYAK&serviceCategoryId=a2ee489a-c7aa-3424-e65a-8fec37beee6c
X-API-Key: pv_live_...

```

`zone` is optional (`&zone=CHENNAI`) for a zone-specific rule; omit it for the role's default rule.

Response — the same `pricing` object embedded in §2.2, on its own:

```

{
  "success": true,
  "data": {
    "configured": true,
    "pricingType": "DYNAMIC",
    "currency": "INR",
    "baseFare": 200.00,
    "hourlyRate": 100.00,
    "perDayRate": null,
    "perMonthRate": null,
    "perKmRate": null,
    "overageRate": null,
    "submissionFee": 0.00,
    "surchargeMultiplier": 1.00,
    "freeTier": 60,
    "billingCap": 150,

```

```

    "workingHoursPerDay": 9,
    "locationZone": null
  },
  "timestamp": "2026-10-01T12:12:00Z"
}

```

How the price is computed

Past the free tier, **DYNAMIC** pricing bills in tiers — full working-days (at **perDayRate**) and full working-months (**perMonthRate**, 30 working days) are peeled off first, with the remainder billed hourly:

```

billable = max(0, min(usage - freeTier, billingCap)) // usage = visit minutes (or km for
distance gigs)
// billingCap limits the BILLABLE amount above the free tier, not the total
fair     = baseFare + (billable × pricing, tiered as above) + submissionFee

```

STANDARD pricing is flatter — everything past the free tier is billed at a single **overageRate** (per minute or per km); **hourlyRate/perDayRate/perMonthRate** don't apply.

fair is what we call the provider's **subtotal**. The amount a patient actually pays adds a platform fee + GST on top, resolved per currency/role at checkout — that step isn't part of this API (it varies by the patient's chosen payment currency).

2.4 Gig forms — PRE / DURING / POST

```

GET /external/v1/gig-forms?role=SAHAYAK&serviceCategoryId=a2ee489a-c7aa-3424-e65a-
8fec37beee6c&stage=PRE
X-API-Key: pv_live_...

```

The full dynamic form set for a service category: every stage section in order, each with its field schema — **identical to what the provider app renders**, including field types, options, validation, units and help text. Omit **stage** for the whole lifecycle (PRE + DURING + POST together); pass **PRE**, **DURING** or **POST** to fetch just one. **locale** (default **en**) selects the i18n language for labels where translated.

Response — no stage param (whole lifecycle):

```

{
  "success": true,
  "data": {
    "serviceCategoryId": "a2ee489a-c7aa-3424-e65a-8fec37beee6c",
    "serviceCategoryName": "Vitals",
    "providerType": "SAHAYAK",
    "actorRole": "SAHAYAK",
    "totalFields": 5,
    "sections": [
      {
        "gigTypeId": "9f1a2b3c-4d5e-6f70-8192-a3b4c5d6e7f8",
        "gigTypeCode": "GIG_NAVIGATION",
        "gigTypeName": "Check-in",
        "stage": "PRE",
        "sortOrder": 1,
        "required": true,
        "form": { "serviceTypeCode": "GIG_NAVIGATION", "fieldGroups": [] }
      },
      {
        "gigTypeId": "1a2b3c4d-5e6f-7081-92a3-b4c5d6e7f809",
        "gigTypeCode": "VITALS_CAPTURE",
        "gigTypeName": "Vitals Capture",

```

```

"stage": "DURING",
"sortOrder": 2,
"required": true,
"form": {
  "serviceTypeCode": "VITALS_CAPTURE",
  "fieldGroups": [
    {
      "categoryName": "Vital Signs",
      "isRepeatable": false,
      "fields": [
        {
          "fieldId": "3c4d5e6f-7081-92a3-b4c5-d6e7f8091a2b",
          "fieldKey": "bp_systolic",
          "label": "Blood Pressure – Systolic",
          "fieldType": "NUMBER",
          "unit": "mmHg",
          "required": true,
          "readOnly": false,
          "autoTimestamp": false,
          "minValue": 60,
          "maxValue": 250
        },
        {
          "fieldId": "4d5e6f70-8192-a3b4-c5d6-e7f8091a2b3c",
          "fieldKey": "pulse",
          "label": "Pulse rate",
          "fieldType": "NUMBER",
          "unit": "bpm",
          "required": true,
          "readOnly": false,
          "autoTimestamp": false
        }
      ]
    },
    {
      "categoryName": "Observation",
      "isRepeatable": false,
      "fields": [
        {
          "fieldId": "5e6f7081-92a3-b4c5-d6e7-f8091a2b3c4d",
          "fieldKey": "mood",
          "label": "Patient mood",
          "fieldType": "SELECT",
          "required": false,
          "readOnly": false,
          "autoTimestamp": false,
          "options": [
            { "value": "CALM", "label": "Calm" },
            { "value": "ANXIOUS", "label": "Anxious" },
            { "value": "DISTRESSED", "label": "Distressed" }
          ]
        }
      ]
    }
  ]
},
{
  "gigTypeId": "2b3c4d5e-6f70-8192-a3b4-c5d6e7f80912",
  "gigTypeCode": "SAHAYAK_SIGNOFF",
  "gigTypeName": "Sign-off",
  "stage": "POST",
  "sortOrder": 3,
  "required": true,
  "form": {
    "serviceTypeCode": "SAHAYAK_SIGNOFF",
    "fieldGroups": [
      {
        "categoryName": "Confirmation",
        "isRepeatable": false,
        "fields": [

```

```

    {
      "fieldId": "6f708192-a3b4-c5d6-e7f8-091a2b3c4d5e",
      "fieldKey": "sign_off_confirmed",
      "label": "Visit completed and confirmed with patient",
      "fieldType": "BOOLEAN",
      "required": true,
      "readOnly": false,
      "autoTimestamp": false
    }
  ]
},
"timestamp": "2026-10-01T12:13:00Z"
}

```

With &stage=PRE — same shape, **sections** filtered to just the PRE section(s).

Field types you'll see in `fieldType`: `TEXT`, `TEXTAREA`, `NUMBER`, `DATE`, `DATETIME`, `BOOLEAN`, `SELECT`, `MULTI_SELECT`, `GEO_LOCATION`, `ATTACHMENT`, `DYNAMIC_DB`, `RATING_STARS`, `SIGNATURE`, among others — render each according to type the same way you'd render any dynamic form; `options[]` is populated for `SELECT/MULTI_SELECT`. There is no write endpoint on this API — your app should collect answers in its own storage/flow; it does not submit back to Preventia.

Full field reference

The examples above trim a field object to what's usually populated. `required`, `readOnly` and `autoTimestamp` are **always present** (`true/false`, never omitted, even when `false`); everything else below is omitted when unset:

Field	Type	Meaning
<code>fieldId</code>	uuid	Stable id of the field definition
<code>fieldKey</code>	string	Machine key — what you'd store the answer under
<code>label</code>	string	Display label, already resolved for the requested <code>locale</code>
<code>fieldType</code>	string	See the type list above
<code>unit</code>	string	e.g. "mmHg", "bpm"
<code>unitOptions</code>	string	CSV of alternate units when more than one is allowed
<code>required</code>	boolean	<i>(always present)</i>
<code>readOnly</code>	boolean	<i>(always present)</i> — e.g. an identity field the app pre-fills, never typed
<code>autoTimestamp</code>	boolean	<i>(always present)</i> — the app stamps the current time into this field at submit; never typed
<code>minValue / maxValue</code>	number	Numeric bounds, where relevant
<code>validationRegex</code>	string	Pattern the answer must match
<code>defaultValue</code>	string	Pre-filled value
<code>placeholder / helpText</code>	string	Input hint / inline explanation
<code>options</code>	array of <code>{value, label}</code>	Choices for <code>SELECT/MULTI_SELECT</code>
<code>icon</code>	string	Ionicons glyph name our apps render next to the field

Field	Type	Meaning
linkTarget	string	For <code>fieldType</code> : "LINK" — which app screen it navigates to
columnSpan	integer	Width on a 12-column grid (12 = full row, 6 = half, 4 = third)
groupLabel	string	Sub-heading this field renders under, if any
groupIsRepeatable	boolean	When true, this field's <code>groupLabel</code> run repeats as chip-added entries (e.g. a medicine list)
children	array of field	Child fields for a repeatable entry row
dependsOn	array of string	Other <code>fieldKeys</code> this field's options/visibility depend on
visibleWhen	object	<code>{field, values[]}</code> — only show this field when a sibling field holds one of <code>values</code>
codeSystem / code / codeDisplay	string	Clinical coding (LOINC/SNOMED/ICD10/RXNORM/CVX), for apps doing FHIR export

`GigFieldGroupDto` (each entry in `fieldGroups`) similarly always includes `"isRepeatable"`: `true|false`, plus `categoryCode/categoryName` when set.

Quick start — curl

```

BASE=https://api.preventia360.com/api/v1
export KEY=pv_live_XXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXX
curl -s -H "X-API-Key: $KEY" "$BASE/external/v1/roles" | jq
curl -s -H "X-API-Key: $KEY" \
  "$BASE/external/v1/service-categories?role=SAHAYAK" | jq
curl -s -H "X-API-Key: $KEY" \
  "$BASE/external/v1/gig-forms?role=SAHAYAK&serviceCategoryId=<uuid>" | jq

```

Operational notes

- The gig-forms response is backed by the same 5-minute Redis cache the provider app reads — a form change in the admin portal shows up here within 5 minutes.
- `GET /external/v1/pricing` (and the `pricing` embedded in `/service-categories`) always reflects the live `service_categories` / `provider_gig_pricing` rows — no cache on the pricing path.
- There is no rate limit enforced today; keep polling reasonable (cache the catalog client-side — service categories and forms rarely change intraday).
- A key has no expiry — it's valid until an admin revokes or deletes it.